

```

def __ne__(self, oOther): # called for != operator
    return not (self == oOther) # calls __eq__ method

def __lt__(self, oOther): # called for < operator
    if abs(self) < abs(oOther): # calls __abs__ method
        return True
    else:
        return False

def __gt__(self, oOther): # called for > operator
    if abs(self) > abs(oOther): # calls __abs__ method
        return True
    else:
        return False

```

Listing 9-8: The `Vector` class that implements a number of magic methods

This class implements arithmetic and comparison operators as magic methods. Client code would use standard symbols for math and comparison between two `Vector` objects. For example, the addition of vectors in [Figure 9-2](#) could be handled like this:

```

oVector1 = Vector(3, 2)
oVector2 = Vector(1, 3)
oNewVector = oVector1 + oVector2 # use the + operator to add vectors

```

When the third line runs, the `__add__()` method ❶ is called to add the two `Vector` objects, resulting in the creation of a new `Vector` object. There is a special check in the `__mul__()` method ❷ that allows the `*` operator to either multiply two `Vectors` or multiply one `Vector` by a scalar value, depending on the type of the second value.

Creating a String Representation of Values in an Object

A standard approach to debugging is to add calls to `print()` to write out the values of variables at certain points in your program: